

European Synchrotron Radiation Facility

Grenoble, France

High Performance Algorithms for Tomography

Optimisation d'une correction de diffusion optique
pour la tomographie X

Juin – août 2026

Stagiaire : Rana ALOUANI

Formation : Licence 3 Physique

Université : Université Grenoble Alpes

Encadrant : Alessandro Mirone

Organisme : European Synchrotron Radiation Facility (ESRF)

Grenoble, 2026

Synthèse

Ce stage a porté sur l'étude et l'optimisation d'une correction de diffusion optique utilisée dans le traitement de données de tomographie X à l'European Synchrotron Radiation Facility.

Dans un système de détection utilisant un scintillateur, les photons X sont convertis en lumière visible. Une partie de cette lumière se propage avant d'atteindre la caméra, ce qui redistribue spatialement le signal produit par l'échantillon.

La correction de cet effet existait déjà dans la chaîne de traitement, mais son application à pleine résolution sur de grandes projections et sur un grand nombre d'angles produisait un coût important en calcul et en stockage.

L'analyse du noyau de diffusion a montré qu'il contient plusieurs échelles spatiales. Les composantes de grande portée varient lentement dans l'espace et peuvent être représentées sur une grille plus grossière. Les contributions plus locales contiennent au contraire davantage d'information haute fréquence et nécessitent une résolution plus fine.

Cette observation a conduit à séparer la correction en deux composantes : une partie basse fréquence calculée à résolution spatiale et angulaire réduite, et une partie haute fréquence appliquée localement.

La basse fréquence a été développée et validée en Python puis stockée sous forme de données HDF5 réutilisables pendant la reconstruction. La haute fréquence a été intégrée dans le code C++ de lecture et de prétraitement des projections, en utilisant FFTW pour réaliser les convolutions locales.

La configuration finale utilise notamment un paramètre de séparation $\sigma = 8$ pixels, un binning spatial de facteur 4 et un espacement angulaire de facteur 4.

Les mesures finales montrent une réduction de l'empreinte disque d'environ 4,7 To à 80 Go, une réduction du temps de création du digest de 32 à 4 minutes, et un passage du temps de reconstruction de 118 à 93 minutes.

Table des matières

Synthèse	1
1 Contexte et problématique	4
1.1 Contexte du stage	4
1.2 Tomographie X	4
1.3 Problématique	4
2 Origine de la diffusion et formation de l'image	5
2.1 Du rayonnement X au signal visible	5
2.2 Modèle par convolution	5
3 Un noyau de diffusion multi-échelle	6
3.1 Profil spatial du noyau	6
4 Pourquoi la correction complète est coûteuse	8
4.1 Déconvolution de référence	8
5 Séparation basse fréquence et haute fréquence	9
5.1 Définition de la correction	9
6 Réduction du coût de la basse fréquence	11
6.1 Réduction spatiale	11
6.2 Réduction angulaire	11
7 Traitement local de la haute fréquence	14
7.1 Exploitation de la localité spatiale	14
8 Pipeline multirésolution complet	16
8.1 Combinaison des deux approximations	16
9 Analyse de l'erreur restante	18
10 Implémentation dans la chaîne de reconstruction	20
10.1 Nouvelle organisation de la correction	20
10.2 Pourquoi utiliser Python pour la basse fréquence et C++ pour la haute fréquence ? . .	20
10.3 Préparation de la basse fréquence en Python	21

10.3.1	Entrées du calcul	21
10.3.2	Calcul et stockage de la correction réduite	21
10.3.3	Réutilisation pendant la reconstruction	22
10.4	Application locale de la haute fréquence en C++	22
10.4.1	Préparation du kernel haute fréquence	22
10.4.2	Chargement dans <code>nabuxx</code>	23
10.4.3	Fonction dédiée à la correction haute fréquence	23
10.4.4	Convolution locale avec FFTW	23
10.5	Flux final des données	24
11	Validation sur reconstruction expérimentale	26
11.1	Effet progressif de la correction	26
11.2	Comparaison avec la correction complète de référence	26
12	Performances mesurées	28
13	Outils utilisés et compétences développées	29
13.1	Environnement et outils utilisés	29
13.1.1	Programmation et calcul scientifique	29
13.1.2	Visualisation et traitement d'images	29
13.1.3	Édition et comparaison de fichiers	29
13.1.4	Environnements de calcul	29
13.1.5	Terminal Linux	30
13.1.6	Versionnement et gestion du code	30
13.2	Compétences développées	30
13.2.1	Relier physique et calcul numérique	30
13.2.2	Construire et valider une méthode progressivement	31
13.2.3	Manipuler des données expérimentales volumineuses	31
13.2.4	Intégrer une solution dans un environnement de production	31
13.2.5	Communication scientifique	31
Conclusion		32
A	Fichiers, scripts et dépôt du projet	33

1 Contexte et problématique

1.1 Contexte du stage

Ce stage de trois mois à l'ESRF a été réalisé volontairement, en dehors d'une obligation de formation.

L'objectif était de prolonger une formation en physique par une expérience plus longue sur un problème scientifique réel, à l'interface entre modélisation physique, traitement numérique des données et développement logiciel.

Le travail a progressivement évolué d'une compréhension du phénomène physique vers l'intégration d'une stratégie d'optimisation dans une base logicielle existante.

1.2 Tomographie X

La tomographie X permet de reconstruire la structure tridimensionnelle d'un échantillon à partir d'un ensemble de projections enregistrées pour différents angles.

La qualité du volume reconstruit dépend directement de la qualité des projections utilisées.

Les imperfections introduites par le système de détection peuvent donc se propager jusqu'au volume reconstruit.

1.3 Problématique

L'une de ces imperfections provient de la diffusion optique associée au système de détection.

La correction de cette diffusion était déjà possible dans la chaîne de traitement.

Le problème étudié pendant le stage était donc le suivant :

Comment conserver la correction de diffusion optique tout en réduisant fortement le coût numérique du traitement à pleine résolution ?

2 Origine de la diffusion et formation de l'image

2.1 Du rayonnement X au signal visible

Dans une expérience de tomographie X, le détecteur ne mesure pas nécessairement directement les photons X incidents.

Un scintillateur convertit l'énergie déposée par les rayons X en photons de lumière visible. Cette lumière est ensuite transmise vers le capteur.

Idéalement, l'information produite en une position donnée devrait rester localisée. Dans un système réel, une partie de la lumière peut cependant se propager avant d'être enregistrée.

Une source ponctuelle idéale n'est donc pas enregistrée comme un point strictement ponctuel : son intensité est redistribuée autour de sa position d'origine.

2.2 Modèle par convolution

Cette redistribution peut être décrite à l'aide d'un noyau de diffusion, ou *kernel*.

En notant I_{true} l'image idéale, K le noyau de diffusion et I_{measured} l'image enregistrée, on peut écrire de manière schématique :

$$I_{\text{measured}} = I_{\text{true}} * K, \quad (2.1)$$

où $*$ représente la convolution.

Le noyau décrit donc la manière dont le signal associé à une source localisée est distribué autour de sa position initiale.

La convolution applique cette même réponse à l'ensemble des pixels de l'image.

3 Un noyau de diffusion multi-échelle

3.1 Profil spatial du noyau

Le noyau réel n'est pas caractérisé par une seule longueur spatiale.

Plusieurs contributions coexistent : certaines restent très proches du centre tandis que d'autres s'étendent sur une distance importante.

Le modèle utilisé décrit le noyau comme la superposition d'une contribution directe et de plusieurs composantes diffuses possédant chacune une échelle spatiale caractéristique.

La distance au centre du noyau est définie par :

$$r = \sqrt{x^2 + y^2}. \quad (3.1)$$

Pour une composante diffuse i , le profil radial peut être écrit sous la forme :

$$K_i(r) \propto F_i \frac{\exp(-r/S_i)}{r}, \quad r > 0. \quad (3.2)$$

où S_i représente la longueur caractéristique de la composante et F_i son poids relatif.

Le noyau total peut alors être représenté schématiquement par :

$$K(r) \propto \delta(r) + \sum_i F_i \frac{\exp(-r/S_i)}{r}. \quad (3.3)$$

où :

- r représente la distance au centre du noyau ;
- S_i est la longueur caractéristique de la composante i ;
- F_i est le poids associé à la composante i ;
- $\delta(r)$ représente la contribution directe non diffusée.

Au voisinage du centre du noyau, lorsque $r \rightarrow 0$, l'expression analytique des composantes diffuses contient un terme en $1/r$ et ne doit donc pas être utilisée directement au point central.

Dans l'implémentation, la contribution diffuse est traitée explicitement au centre afin d'éviter cette singularité numérique. La contribution directe au point $r = 0$ est représentée séparément par le terme $\delta(r)$.

La divergence apparente de l'expression analytique en $r = 0$ ne doit donc pas être interprétée comme une divergence physique du signal, mais comme une conséquence de la forme utilisée pour décrire les composantes diffuses à distance non nulle du centre.

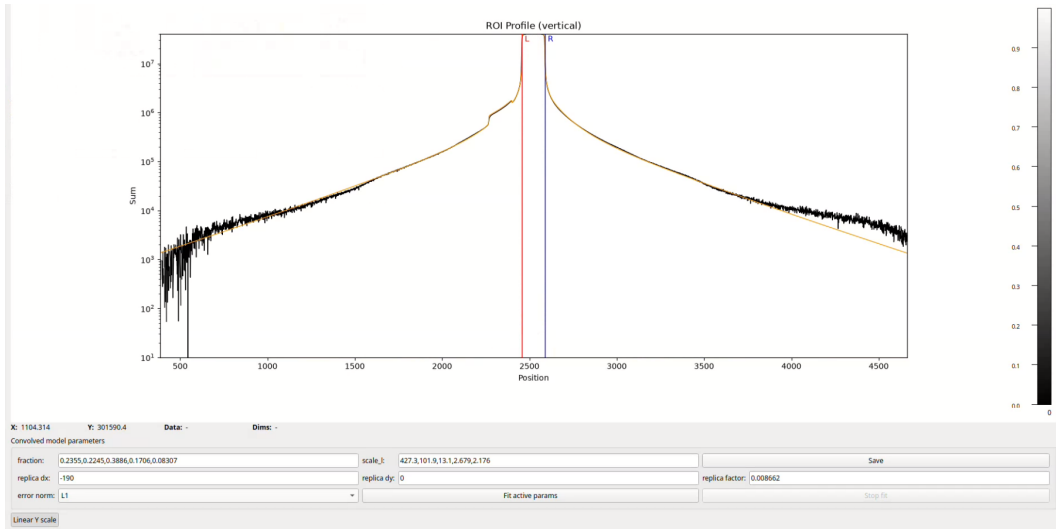


FIGURE 3.1 – Profil vertical du noyau de diffusion réel en échelle logarithmique. Le profil met en évidence la coexistence de contributions locales autour du centre et de composantes de plus longue portée formant un halo spatial étendu.

La figure 3.1 montre directement que le phénomène est multi-échelle.

Les composantes de courte portée produisent des variations très locales, tandis que les composantes de longue portée donnent naissance au halo étendu.

Dans le modèle utilisé pour caractériser la diffusion, cette structure multi-échelle est décrite par plusieurs longueurs caractéristiques.

Les valeurs du paramètre `scale_1` correspondent au vecteur :

$$\mathbf{S} = (115.0, 34.63, 7.414, 3.472, 1.016) . \quad (3.4)$$

Les fractions associées aux différentes contributions correspondent au vecteur :

$$\mathbf{F} = (0.1859, 0.1590, 0.1196, 0.1375, 0.1971) . \quad (3.5)$$

Chaque couple (S_i, F_i) correspond ainsi à une composante du noyau caractérisée par une portée spatiale et un poids différents.

Les grandes valeurs de S_i décrivent les contributions les plus étendues, tandis que les petites valeurs correspondent à des contributions plus localisées autour du centre.

Les valeurs de `scale_1` ne doivent pas être interprétées comme des largeurs gaussiennes σ . Elles représentent les longueurs caractéristiques des différentes composantes du modèle de diffusion.

La coexistence de ces différentes échelles spatiales constitue le point de départ de la stratégie d'optimisation développée dans la suite de ce travail.

4 Pourquoi la correction complète est coûteuse

4.1 Déconvolution de référence

Pour corriger la diffusion, la méthode de référence réalise une déconvolution.

Dans le domaine de Fourier, on peut écrire de manière schématique :

$$\hat{I}_{\text{corr}} = \frac{\hat{I}_{\text{meas}}}{\hat{K}}, \quad (4.1)$$

où :

- $\hat{I}_{\text{corr}}$ est la transformée de Fourier de la projection corrigée ;
- $\hat{I}_{\text{meas}}$ est la transformée de la projection mesurée ;
- $\hat{K}$ est la transformée de Fourier du noyau de diffusion.

En pratique, cette correction nécessite plusieurs opérations successives. Chaque projection est d'abord étendue par padding afin de limiter les effets de bord. Une transformée de Fourier est ensuite calculée afin de passer dans le domaine fréquentiel, où la correction est appliquée à partir du noyau de diffusion. Une transformée de Fourier inverse permet ensuite de revenir dans l'espace réel, avant de recadrer l'image pour retrouver la zone utile de la projection.

Cette procédure devient coûteuse car elle doit être répétée pour un très grand nombre de projections, chacune étant traitée à pleine résolution. Chaque projection nécessite donc des opérations de padding, de transformée de Fourier, de correction dans le domaine fréquentiel puis de transformée inverse sur des tableaux de grande taille. À l'échelle d'une acquisition tomographique complète, cela représente un volume important de données à manipuler et un coût de calcul élevé.

Une question se pose alors : si une partie de la correction varie seulement lentement dans l'espace et entre des projections voisines, est-il réellement nécessaire de la calculer partout et à pleine résolution ?

L'idée consiste donc à ne plus traiter l'ensemble de la correction de la même manière, mais à introduire une **séparation basse fréquence / haute fréquence** : une composante basse fréquence, lente et étendue, et une composante haute fréquence, plus locale et détaillée.

5 Séparation basse fréquence et haute fréquence

5.1 Définition de la correction

On définit tout d'abord la correction Q comme la différence entre le signal mesuré et le signal entièrement corrigé :

$$Q = I_{\text{meas}} - I_{\text{corr}}, \quad (5.1)$$

où I_{meas} désigne la projection mesurée par le détecteur et I_{corr} la projection obtenue après application de la correction complète de référence.

L'idée consiste ensuite à ne plus traiter cette correction comme un seul objet, mais à la séparer en deux contributions complémentaires : une composante basse fréquence et une composante haute fréquence.

La composante basse fréquence est obtenue en appliquant un filtre gaussien à la correction :

$$Q_{\text{LF}} = G_{\sigma} * Q, \quad (5.2)$$

où G_{σ} désigne un filtre gaussien de largeur σ .

Dans le travail présenté ici, la valeur

$$\sigma = 8 \text{ pixels} \quad (5.3)$$

a été retenue. Cette largeur permet de conserver dans Q_{LF} les variations lentes que l'on souhaite traiter comme une composante globale, tout en séparant suffisamment les structures plus locales. Le choix de σ fixe donc la frontière entre les deux échelles de correction.

La composante haute fréquence n'est pas calculée indépendamment. Elle est simplement définie comme ce qui reste après soustraction de la basse fréquence :

$$Q_{\text{HF}} = Q - Q_{\text{LF}}. \quad (5.4)$$

Par construction, les deux contributions redonnent donc exactement la correction initiale :

$$Q = Q_{\text{LF}} + Q_{\text{HF}}. \quad (5.5)$$

Cette propriété est importante : la séparation ne modifie pas à ce stade la correction de référence. Elle constitue uniquement une décomposition de celle-ci en deux contributions présentant des propriétés spatiales différentes.

La composante basse fréquence contient principalement les variations lentes et spatialement étendues, tandis que la composante haute fréquence conserve les structures plus localisées.

La même construction peut être appliquée au **noyau de la correction**. C'est cette décomposition du noyau qui est représentée sur la figure 5.1. La figure ne montre donc pas directement la décomposition

d'une projection corrigée, mais la séparation du noyau de correction en une contribution basse fréquence et une contribution haute fréquence.

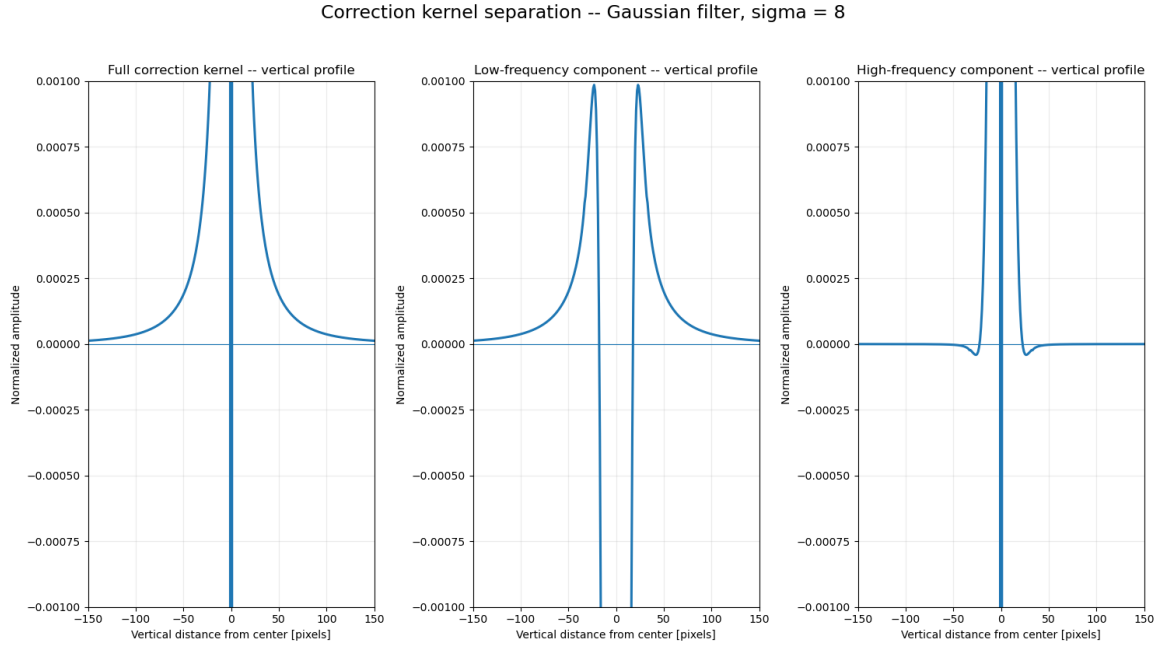


FIGURE 5.1 – Décomposition du noyau de correction en une composante basse fréquence et une composante haute fréquence pour $\sigma = 8$ pixels. La composante basse fréquence conserve la structure spatialement étendue du noyau, tandis que la composante haute fréquence est davantage concentrée autour de son centre. Les échelles d’affichage sont adaptées indépendamment afin de rendre visible la composante haute fréquence.

Cette décomposition fait apparaître deux propriétés différentes qui peuvent être exploitées numériquement : la basse fréquence est lisse et peut potentiellement être représentée à plus faible résolution, tandis que la haute fréquence est plus localisée et peut potentiellement être traitée sur une région limitée. La stratégie d’optimisation consiste donc à adapter le coût du calcul aux propriétés propres de chacune de ces deux composantes.

6 Réduction du coût de la basse fréquence

6.1 Réduction spatiale

La première optimisation concerne la composante basse fréquence. Comme elle varie lentement dans l'espace, il n'est pas nécessaire de la représenter avec la même densité de pixels que la correction complète.

On peut donc commencer par réduire sa résolution spatiale avant de la traiter.

Pour une projection de dimensions $N_x \times N_y$, un binning spatial de facteur b dans les deux directions conduit à une représentation de dimensions :

$$\frac{N_x}{b} \times \frac{N_y}{b}. \quad (6.1)$$

Le nombre total d'échantillons spatiaux devient alors :

$$\frac{N_x N_y}{b^2}. \quad (6.2)$$

Avec $b = 4$, cela correspond à une division par 16 du nombre d'échantillons spatiaux.

Ce choix reste compatible avec la séparation fréquentielle utilisée ici. Pour $\sigma = 8$ pixels, les structures conservées dans la composante basse fréquence sont déjà lissées sur plusieurs pixels. Après un binning spatial de facteur 4, cette échelle caractéristique correspond encore à environ deux pixels dans l'image réduite. La taille des données peut donc être fortement diminuée sans supprimer les variations lentes que l'on cherche précisément à conserver.

Cette réduction d'un facteur 16 concerne cependant uniquement le nombre d'échantillons spatiaux. Elle ne signifie pas que l'algorithme complet est automatiquement 16 fois plus rapide. Le gain réel dépend de l'ensemble de l'implémentation, notamment du coût des FFT, des accès mémoire et des entrées-sorties.

6.2 Réduction angulaire

La réduction spatiale n'est toutefois pas la seule possibilité. La composante basse fréquence évolue également relativement lentement d'une projection à la suivante. Cette propriété suggère une seconde réduction, cette fois dans la direction angulaire : au lieu de calculer directement la basse fréquence pour chaque projection, il devient possible de ne la calculer que pour certaines projections puis d'interpoler les valeurs intermédiaires.

Pour vérifier cette idée, un test a été réalisé sur la projection 102. La correction basse fréquence de cette projection n'est pas utilisée pour construire la prédiction. Elle est conservée uniquement comme référence. Les corrections basse fréquence sont calculées pour les projections 100 et 104, puis réduites spatialement d'un facteur 4. La valeur correspondant à la projection 102 est ensuite obtenue par interpolation entre ces deux projections voisines.

Dans ce cas, le poids d'interpolation vaut :

$$t = \frac{102 - 100}{104 - 100} = \frac{1}{2}. \quad (6.3)$$

L'interpolation tient également compte d'une normalisation par le courant du faisceau, car l'intensité globale peut varier d'une projection à l'autre.

Afin de comparer cette approximation à la référence, la figure 6.1 présente à gauche la véritable correction basse fréquence de la projection 102, au centre la correction prédite à partir des projections 100 et 104, et à droite leur différence.

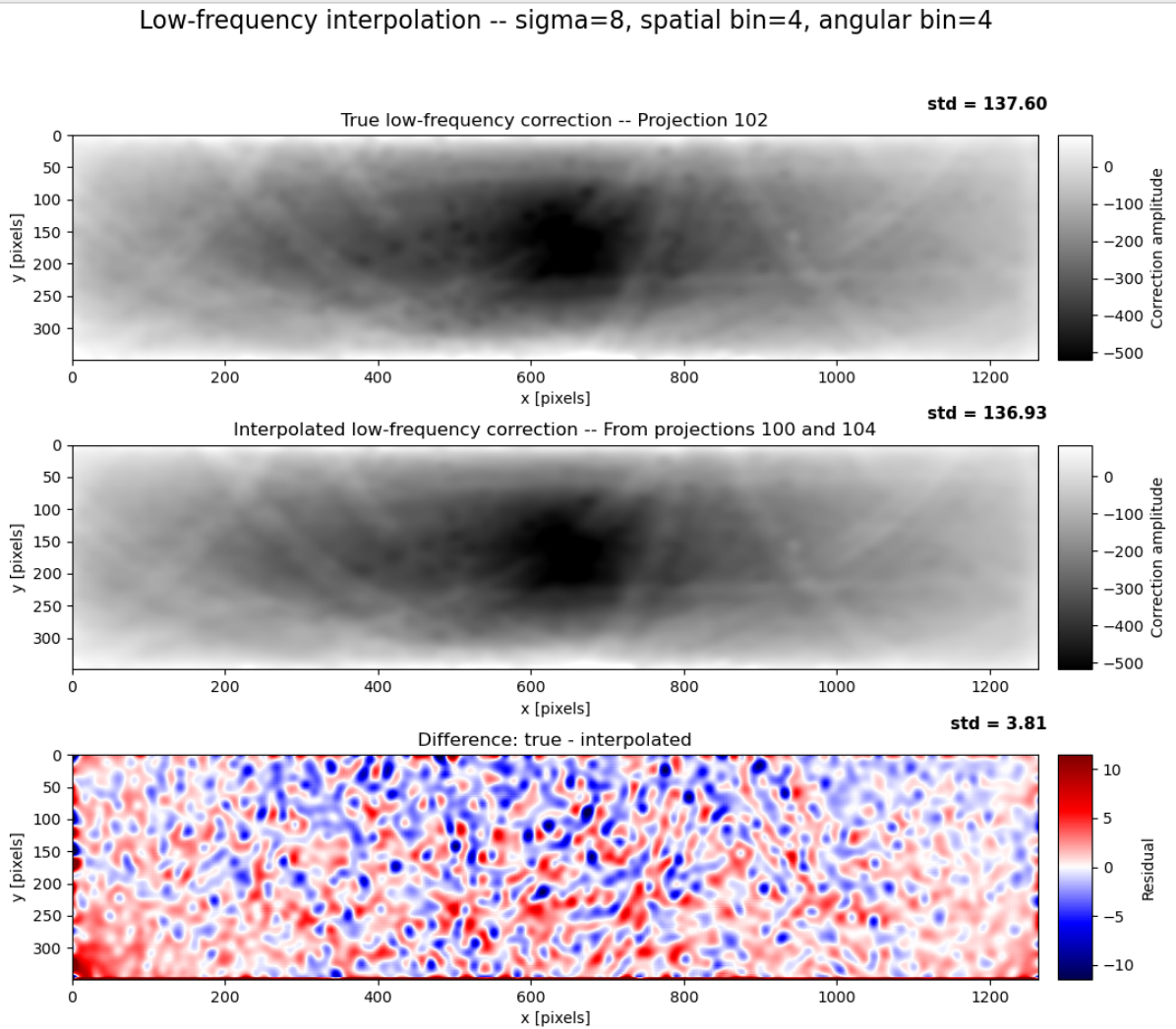


FIGURE 6.1 – Validation de la réduction spatiale et angulaire de la composante basse fréquence pour la projection 102. À gauche : correction basse fréquence de référence. Au centre : prédiction obtenue à partir des projections 100 et 104 avec un binning spatial de facteur 4 et une interpolation angulaire. À droite : résidu entre la référence et la prédiction. La carte du résidu utilise une échelle divergente centrée autour de zéro.

Les structures de grande échelle sont visuellement très proches entre la référence et la prédiction. Le résidu peut sembler marqué en raison de l'échelle de couleurs utilisée, mais son amplitude reste faible par rapport à celle de la correction basse fréquence elle-même.

Les écarts-types mesurés sont :

$$\text{std}(Q_{\text{LF,true}}) = 137.6, \quad (6.4)$$

$$\text{std}(Q_{\text{LF,interp}}) = 136.9, \quad (6.5)$$

$$\text{std}(\text{residual}) = 3.81. \quad (6.6)$$

L'écart-type du résidu représente donc environ 2.8 % de celui de la correction basse fréquence de référence.

Ce test montre que la composante lisse peut être approximée en combinant une réduction spatiale et une réduction angulaire, tout en conservant les structures basse fréquence que l'on souhaite préserver. Cette validation justifie ensuite l'utilisation de ces deux réductions dans la configuration finale du pipeline.

7 Traitement local de la haute fréquence

7.1 Exploitation de la localité spatiale

Contrairement à la composante basse fréquence, la composante haute fréquence ne peut pas être fortement réduite spatialement. Elle contient précisément les structures fines que l'on souhaite préserver, et un binning trop important supprimerait une partie de cette information.

L'optimisation doit donc exploiter une autre propriété de cette composante : sa **localité spatiale**.

La figure 7.1 représente la magnitude normalisée du noyau de correction haute fréquence en fonction de la distance verticale à son centre. L'axe vertical est représenté en échelle logarithmique afin de rendre visibles des amplitudes qui s'étendent sur plusieurs ordres de grandeur.

Au voisinage du centre, l'amplitude de la composante haute fréquence est importante. En revanche, lorsqu'on s'éloigne du centre du noyau, cette amplitude décroît très rapidement. Le graphe montre donc que l'essentiel de la contribution haute fréquence est concentré dans une région spatiale relativement limitée.

Cette observation est particulièrement intéressante du point de vue numérique. Si la contribution haute fréquence devient négligeable loin du centre, il n'est pas nécessaire de la calculer sur toute la hauteur de la projection. Il suffit de conserver une marge autour de la région qui doit effectivement être corrigée.

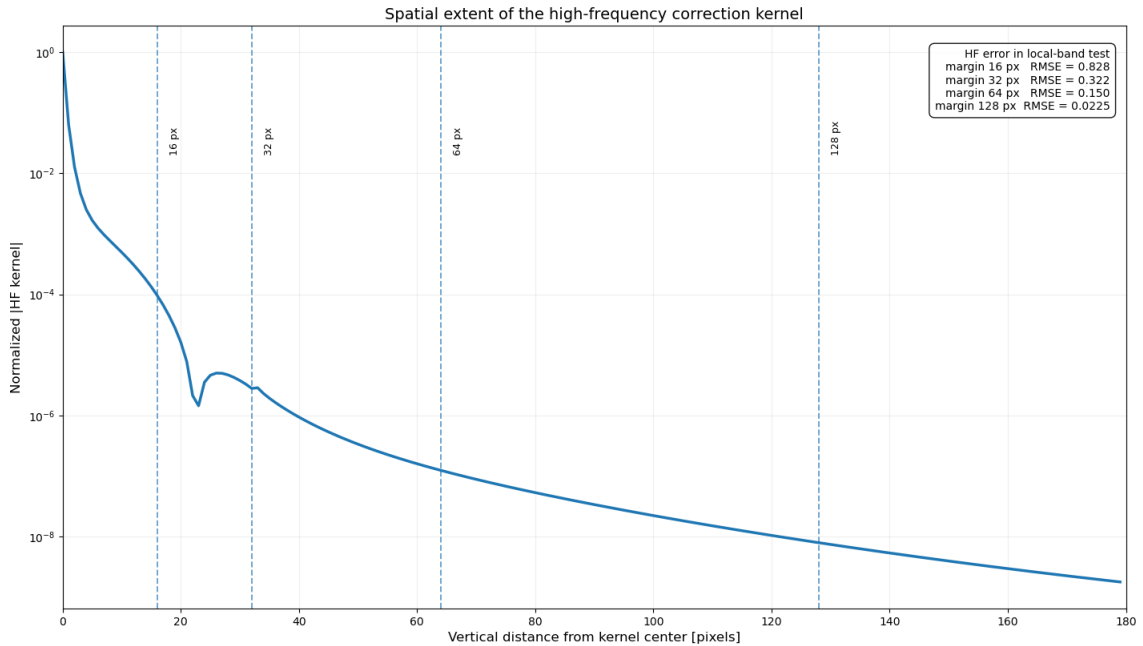


FIGURE 7.1 – Magnitude normalisée du noyau de correction haute fréquence en fonction de la distance verticale à son centre, représentée en échelle logarithmique. L'amplitude décroît de plusieurs ordres de grandeur lorsque l'on s'éloigne du centre. Les différentes marges testées permettent d'évaluer la taille de voisinage nécessaire pour reproduire correctement la contribution haute fréquence.

Afin de déterminer jusqu'à quelle distance cette contribution doit être conservée, plusieurs marges locales ont été testées autour de la région utile.

Les erreurs obtenues pour la composante haute fréquence sont résumées dans le tableau suivant :

Marge locale	RMSE haute fréquence
16 px	0.828
32 px	0.322
64 px	0.150
128 px	0.0225

Avec une marge de 16 pixels, l'erreur reste encore relativement importante, avec une RMSE d'environ 0.83. Lorsque la marge est portée à 32 pixels, l'erreur diminue à environ 0.32. À 64 pixels, elle atteint environ 0.15, puis seulement 0.0225 avec une marge de 128 pixels.

Cette diminution rapide confirme quantitativement ce que suggère le profil du noyau : la majeure partie de la contribution haute fréquence est concentrée près du centre.

Il n'est donc pas nécessaire de traiter toute la hauteur de la projection pour retrouver correctement la haute fréquence. Un voisinage local autour de la région utile suffit à en capturer l'essentiel.

8 Pipeline multirésolution complet

8.1 Combinaison des deux approximations

À ce stade, les deux idées d'optimisation ont été validées séparément. D'une part, la composante basse fréquence peut être approchée à résolution spatiale réduite et interpolée angulairement. D'autre part, la composante haute fréquence peut être calculée localement, car l'essentiel de sa contribution est concentré dans un voisinage limité autour de la région utile.

L'étape suivante consiste donc à combiner ces deux approximations pour reconstruire une approximation de la correction complète.

On définit ainsi :

$$Q_{\text{approx}} = Q_{\text{LF,pred}} + Q_{\text{HF,local}}, \quad (8.1)$$

où :

- $Q_{\text{LF,pred}}$ est la composante basse fréquence prédite à partir d'un calcul à résolution réduite, avec binning spatial et interpolation angulaire ;
- $Q_{\text{HF,local}}$ est la composante haute fréquence calculée localement autour de la région utile.

La projection corrigée approchée s'écrit alors :

$$I_{\text{corr,approx}} = I_{\text{meas}} - Q_{\text{approx}}. \quad (8.2)$$

Dans la configuration étudiée ici, la composante basse fréquence est obtenue avec un binning spatial de facteur 4 et un binning angulaire de facteur 4, tandis que la composante haute fréquence est calculée avec une marge locale de 64 pixels.

La figure 8.1 montre comment ces deux approximations sont combinées sur le cas de la projection 102. En haut à gauche apparaît la véritable correction basse fréquence de référence pour cette projection, tandis qu'en haut à droite figure sa prédiction obtenue à partir des projections voisines avec réduction spatiale et angulaire. En bas à gauche est représentée la correction haute fréquence calculée localement avec une marge de 64 pixels. L'addition de ces deux contributions fournit alors l'approximation complète de la correction. Enfin, le panneau en bas à droite montre le résidu final par rapport à la correction de référence.

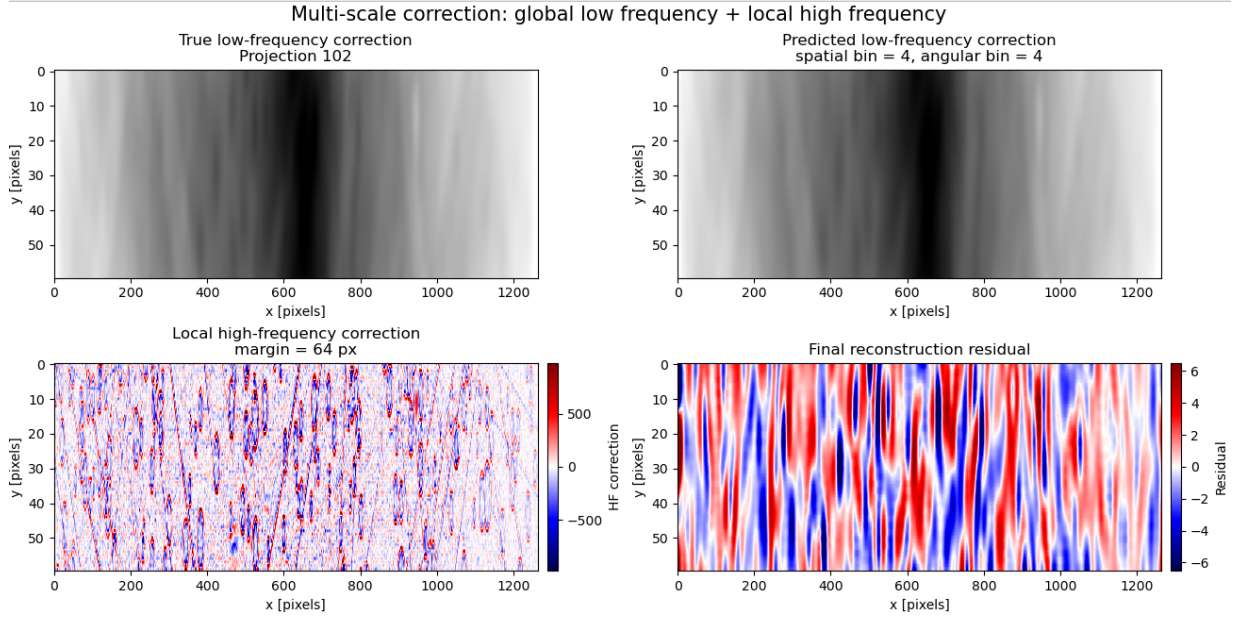


FIGURE 8.1 – Combinaison des deux approximations dans le pipeline multirésolution. En haut à gauche : correction basse fréquence de référence pour la projection 102. En haut à droite : prédiction basse fréquence obtenue avec un binning spatial de facteur 4 et un binning angulaire de facteur 4. En bas à gauche : correction haute fréquence calculée localement avec une marge de 64 pixels. En bas à droite : résidu final entre l’approximation complète et la correction de référence.

Le point important est que le résidu final apparaît beaucoup plus lisse que la composante haute fréquence elle-même. Cela signifie que l’erreur liée à la localisation de la haute fréquence a déjà été fortement réduite.

Cette observation est cohérente avec les résultats quantitatifs obtenus précédemment. Pour une marge de 64 pixels, l’erreur propre à la localisation de la haute fréquence est caractérisée par une RMSE d’environ 0.15, tandis que l’erreur finale de reconstruction de la correction complète est d’environ 2.23.

Autrement dit, à ce stade, la limitation principale ne provient plus de la troncature locale de la haute fréquence. L’erreur restante est dominée par l’approximation de la composante basse fréquence, c’est-à-dire par la réduction spatiale et l’interpolation angulaire utilisées pour prédire $Q_{LF,pred}$.

Cette étape montre donc que les deux contributions peuvent être recombinaées de manière cohérente, et que la stratégie multirésolution permet de reproduire la correction complète avec une erreur principalement contrôlée par la branche basse fréquence.

9 Analyse de l'erreur restante

Les résultats quantitatifs permettent d'identifier plus précisément quelle composante limite désormais la précision de la méthode.

Lorsque la marge utilisée pour le calcul de la composante haute fréquence augmente, l'approximation de cette branche s'améliore fortement. Une marge plus grande permet en effet de conserver une portion plus importante du noyau haute fréquence et donc de réduire l'erreur associée à sa localisation.

Les valeurs obtenues sont résumées dans le tableau suivant :

Marge HF	RMSE HF	RMSE finale
16 px	0.828	2.349
32 px	0.322	2.243
64 px	0.150	2.231
128 px	0.0225	2.226
192 px	0	2.226

La RMSE associée à la haute fréquence diminue donc très rapidement lorsque la marge augmente : elle passe d'environ 0.83 pour une marge de 16 pixels à 0.15 pour 64 pixels, puis devient presque nulle lorsque l'on utilise une région suffisamment grande.

En revanche, l'erreur finale évolue beaucoup moins. Elle passe de 2.349 pour une marge de 16 pixels à environ 2.23 dès que la marge atteint 64 pixels, puis reste pratiquement constante lorsque la marge est encore augmentée.

Au-delà d'environ 64 à 128 pixels, augmenter davantage la marge haute fréquence n'apporte donc presque plus d'amélioration au résultat final.

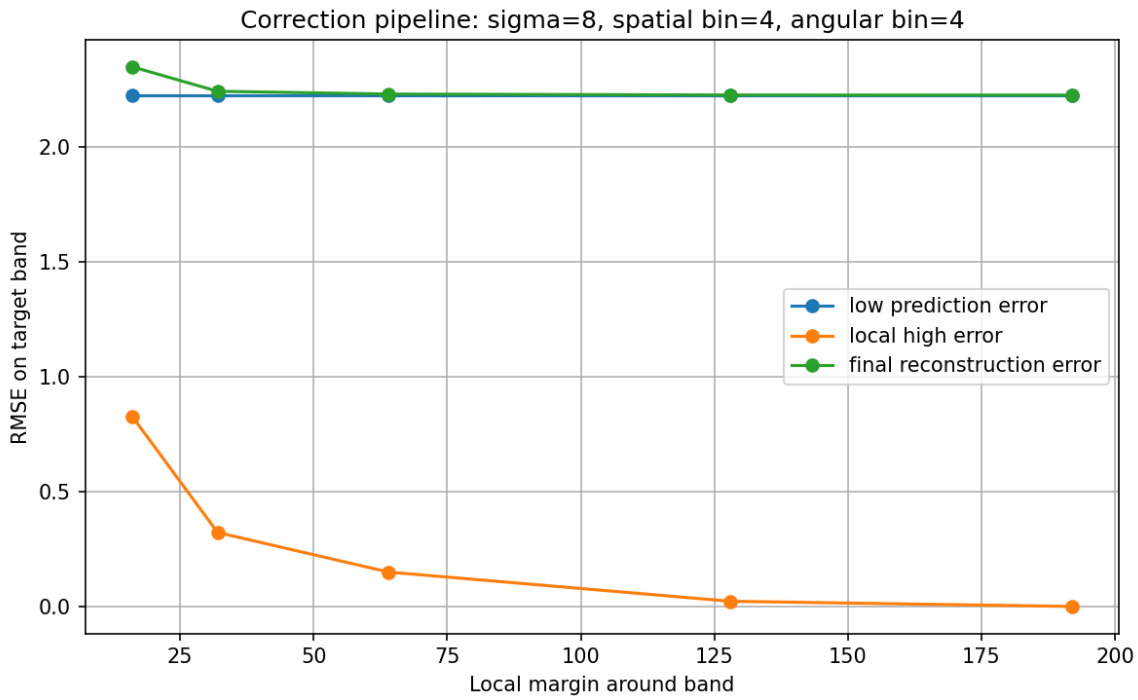


FIGURE 9.1 – Évolution de l’erreur en fonction de la marge utilisée pour le calcul local de la composante haute fréquence. L’erreur haute fréquence diminue fortement lorsque la marge augmente, tandis que l’erreur finale se stabilise autour de 2.23 à partir d’une marge d’environ 64 pixels.

Ce comportement permet d’identifier l’origine de l’erreur restante. Lorsque la marge haute fréquence devient suffisamment grande, l’erreur liée à la localisation de cette composante devient faible devant l’erreur totale.

La RMSE finale, proche de 2.23, est alors très proche de l’erreur associée à la prédiction basse fréquence, d’environ 2.226 dans ce test. Le facteur limitant ne se situe donc plus dans la branche haute fréquence, mais dans l’approximation de la composante basse fréquence.

Autrement dit, la localisation de la haute fréquence fonctionne suffisamment bien. Une amélioration supplémentaire de la précision du pipeline devra désormais porter principalement sur la prédiction basse fréquence, notamment sur la réduction spatiale et l’interpolation angulaire.

10 Implémentation dans la chaîne de reconstruction

10.1 Nouvelle organisation de la correction

La séparation entre basse et haute fréquence ne modifie pas seulement la manière de calculer la correction. Elle permet également de modifier l'endroit où chaque contribution est appliquée dans la chaîne de reconstruction.

Dans l'approche précédente, la correction complète devait être réalisée avant la création du digest. Sa composante de longue portée nécessite en effet des informations provenant d'une région importante de la radiographie, ce qui impose de travailler sur des données beaucoup plus étendues que la seule zone finalement reconstruite.

Après séparation des échelles, cette contrainte ne concerne plus toute la correction. La composante basse fréquence reste globale et doit donc être préparée en amont. À l'inverse, la composante haute fréquence est suffisamment localisée pour être appliquée plus tard, directement pendant le traitement du volume, sur la région déjà disponible dans la marge de lecture.

La nouvelle organisation est donc la suivante :

Basse fréquence : préparation en amont à résolution réduite

+

Haute fréquence : application locale pendant la lecture

Cette organisation permet d'éviter de produire et de stocker une correction complète à pleine résolution pour toutes les projections.

10.2 Pourquoi utiliser Python pour la basse fréquence et C++ pour la haute fréquence ?

Les deux composantes n'interviennent pas au même moment et n'ont pas les mêmes contraintes numériques. Il n'était donc pas nécessaire de les implémenter dans le même environnement.

La composante basse fréquence correspond à une étape de préparation des données. Elle doit construire le modèle de diffusion, manipuler les paramètres expérimentaux, effectuer les opérations de réduction spatiale et angulaire, puis produire des données intermédiaires qui seront réutilisées pendant la reconstruction.

Python est bien adapté à cette phase grâce aux outils de calcul scientifique déjà utilisés pendant le développement, notamment NumPy, SciPy, h5py et pyFFTW. Il permet également de conserver une implémentation proche des scripts utilisés pendant la phase de prototypage.

La composante haute fréquence répond à une contrainte différente. Elle doit être appliquée au moment où les projections sont déjà lues par `nabxxx` et présentes en mémoire. L'intégrer directement dans le code C++ évite donc de créer une nouvelle étape intermédiaire ou de réécrire les projections sur disque avant la reconstruction.

Le choix Python/C++ ne correspond ainsi pas simplement à une préférence de langage : il suit la

position naturelle de chaque composante dans le pipeline.

10.3 Préparation de la basse fréquence en Python

10.3.1 Entrées du calcul

Le programme Python principal utilisé pour cette étape est :

`diffusion_correction.py`

Il utilise les projections expérimentales, le dark et les paramètres du modèle de diffusion. Les principaux fichiers d'entrée sont notamment :

- `projections.h5`;
- `dark.h5`;
- `diffusion_parameters.json`.

Le programme récupère les dimensions des projections ainsi que le courant associé à chaque image. Il applique ensuite les facteurs de réduction spatiale et angulaire définis précédemment.

Par exemple, les dimensions réduites sont calculées dans le programme à partir des facteurs `bin_proj` et `bin_x` :

```
bin_proj = args.diffusion_bin_proj
bin_x = args.diffusion_bin_x

nprojs_binned = (nprojs + bin_proj - 1) // bin_proj
dim_z_binned = (dim_z + bin_x - 1) // bin_x
dim_x_binned = (dim_x + bin_x - 1) // bin_x
```

Cette étape traduit directement dans le code la réduction spatiale et angulaire étudiée précédemment.

10.3.2 Calcul et stockage de la correction réduite

La correction n'est donc calculée que pour les projections retenues par le binning angulaire et sur une grille spatiale réduite.

La composante basse fréquence obtenue doit ensuite rester disponible pour la reconstruction. Elle est enregistrée dans le fichier :

`diffusion_correction.h5`

Le fichier contient notamment trois datasets :

- `difference`, qui contient les valeurs de correction ;
- `binnings`, qui conserve les facteurs de réduction utilisés ;
- `current`, qui conserve le courant associé aux projections.

La création de ces datasets apparaît directement dans le code :

```
f_out.create_dataset(
    "difference",
    expected_shape,
    dtype="f4"
)

f_out.create_dataset(
    "binnings",
    data=np.asarray([bin_proj, bin_x], dtype="i4")
)

f_out.create_dataset(
    "current",
    shape=(nprojs_binned,),
    dtype="f4"
)
```

Le fichier HDF5 constitue ainsi l'interface entre la phase de préparation Python et la reconstruction réalisée ensuite par `nabuxx`.

10.3.3 Réutilisation pendant la reconstruction

Lorsqu'un bloc de projections doit être traité, il n'est pas nécessaire de charger toute la correction basse fréquence en mémoire.

`nabuxx` récupère uniquement la portion du volume de correction nécessaire au bloc actuellement traité. Les valeurs enregistrées à résolution réduite sont ensuite interpolées suivant les dimensions nécessaires pour retrouver la résolution des projections utilisées par la reconstruction.

L'interpolation intervient donc suivant trois directions :

- l'indice de projection ;
- la direction verticale ;
- la direction horizontale.

Cette étape relie la représentation économique construite en Python aux données brutes actuellement présentes dans le pipeline.

10.4 Application locale de la haute fréquence en C++

10.4.1 Préparation du kernel haute fréquence

La haute fréquence doit être appliquée dans le code C++, mais il n'est pas utile de reconstruire le modèle physique du kernel à chaque lecture d'une projection.

Le kernel haute fréquence est donc préparé une seule fois en Python puis enregistré dans :

`kernel.h5`

Ce fichier contient un dataset bidimensionnel nommé :

`kernel`

Le C++ peut ainsi charger directement le kernel déjà calculé et se concentrer uniquement sur son application aux données.

10.4.2 Chargement dans nabuxx

Le kernel est chargé au démarrage de la correction à partir du fichier HDF5. Le code utilise HighFive pour accéder au dataset :

```
h5::File kernel_file(  
    (fs::path(mypath) / "kernel.h5").string(),  
    h5::File::ReadOnly  
);  
  
h5::DataSet kernel_dataset =  
    kernel_file.getDataSet("kernel");
```

Ses dimensions sont vérifiées avant d'être copiées dans la structure utilisée par le calcul C++.

10.4.3 Fonction dédiée à la correction haute fréquence

La correction locale a été isolée dans une fonction dédiée dans `etfscan.cpp` :

```
void apply_high_frequency_convolution(  
    xt::xtensor<float, 3>& raw_data,  
    const xt::xarray<float>& kernel  
)
```

Cette fonction reçoit deux éléments :

- le bloc de projections actuellement présent dans `raw_data`;
- le kernel haute fréquence pré-calculé.

Son rôle est volontairement limité : appliquer la contribution haute fréquence aux données déjà chargées en mémoire puis réinjecter le résultat dans le même bloc.

Cette séparation permet de conserver une fonction identifiable et testable, sans mélanger directement le calcul de convolution avec toute la logique de lecture des fichiers HDF5.

10.4.4 Convolution locale avec FFTW

La convolution haute fréquence est réalisée à l'aide de FFTW.

Une convolution bidimensionnelle directe entre une projection et un kernel peut nécessiter un grand nombre d'opérations. Le calcul est donc réalisé dans le domaine de Fourier en utilisant le théorème de convolution :

$$I * K = \mathcal{F}^{-1} [\mathcal{F}(I) \mathcal{F}(K)], \quad (10.1)$$

où I représente la région de projection traitée et K le kernel haute fréquence.

Le travail réalisé ne consistait pas à réimplémenter l'algorithme FFT lui-même. FFTW fournit les transformées nécessaires. L'intégration porte principalement sur la préparation des données, les dimensions de padding, la gestion des buffers, la multiplication des spectres, la transformée inverse et le recadrage du résultat.

Pour une image de dimensions $H_I \times W_I$ et un kernel de dimensions $H_K \times W_K$, les dimensions nécessaires à la convolution linéaire sont :

$$H_P = H_I + H_K - 1, \quad (10.2)$$

$$W_P = W_I + W_K - 1. \quad (10.3)$$

Dans le code :

```
const std::size_t padded_height =
    image_height + kernel_height - 1;

const std::size_t padded_width =
    image_width + kernel_width - 1;
```

Après la transformée inverse, le résultat est normalisé puis recadré afin de retrouver les dimensions attendues par la suite du pipeline.

10.5 Flux final des données

L'implémentation finale peut donc être résumée de la manière suivante.

Les fichiers `projections.h5`, `dark.h5` et `diffusion_parameters.json` sont d'abord utilisés par le programme Python pour préparer la composante basse fréquence à résolution spatiale et angulaire réduite.

Cette correction est sauvegardée dans :

`diffusion_correction.h5.`

En parallèle, le kernel haute fréquence est pré-calculé en Python et enregistré dans :

`kernel.h5.`

Pendant la reconstruction, `nabuxx` charge uniquement la partie nécessaire de la basse fréquence, l'interpole pour retrouver les valeurs correspondant au bloc de projections courant, puis applique localement la haute fréquence directement en C++.

On obtient ainsi le flux suivant :

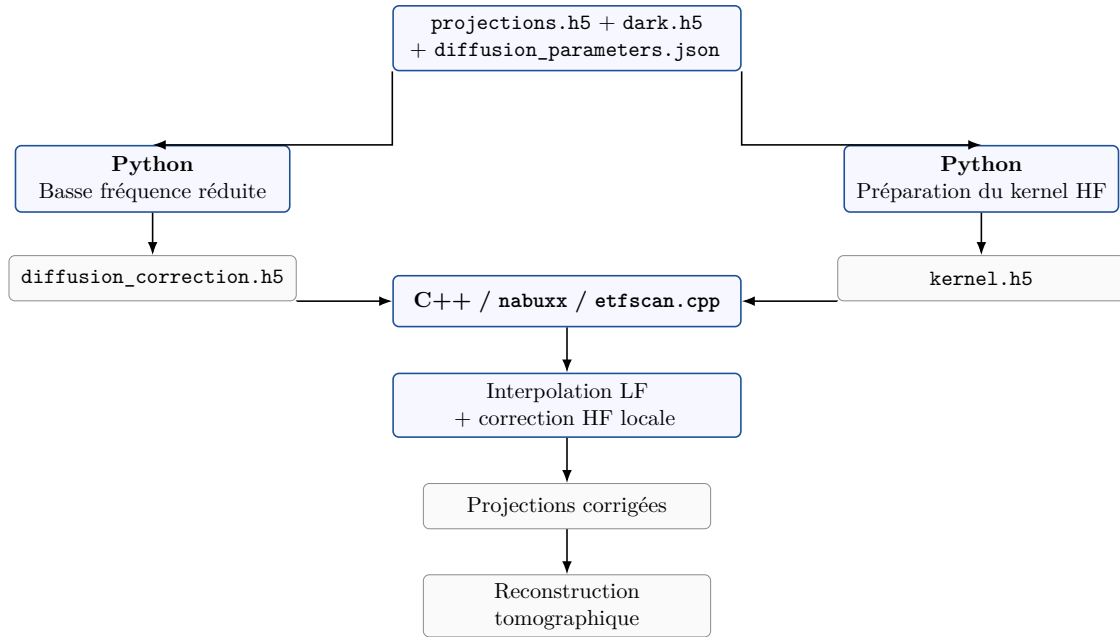


FIGURE 10.1 – Organisation du pipeline après séparation basse fréquence / haute fréquence. La basse fréquence est préparée en Python à résolution réduite et stockée dans `diffusion_correction.h5`. Le kernel haute fréquence est pré-calculé séparément et stocké dans `kernel.h5`. Les deux éléments sont ensuite utilisés dans `nabuxx`, où la basse fréquence est interpolée et la correction haute fréquence appliquée localement avant la reconstruction.

Cette architecture associe donc chaque composante au traitement numérique le mieux adapté à ses propriétés : une préparation réduite et réutilisable pour la basse fréquence, et une application locale directement sur les données en mémoire pour la haute fréquence.

11 Validation sur reconstruction expérimentale

Après avoir validé séparément les différentes composantes de la correction, il reste à vérifier leur effet sur le résultat final de la reconstruction tomographique.

Toutes les images présentées dans cette section correspondent à la **même coupe reconstruite**, avec le **même recadrage** et la **même plage de niveaux de gris**. Elles peuvent donc être comparées directement sans que les différences observées proviennent d'un changement d'affichage.

11.1 Effet progressif de la correction

La première comparaison permet de visualiser progressivement l'effet des différentes étapes de correction.

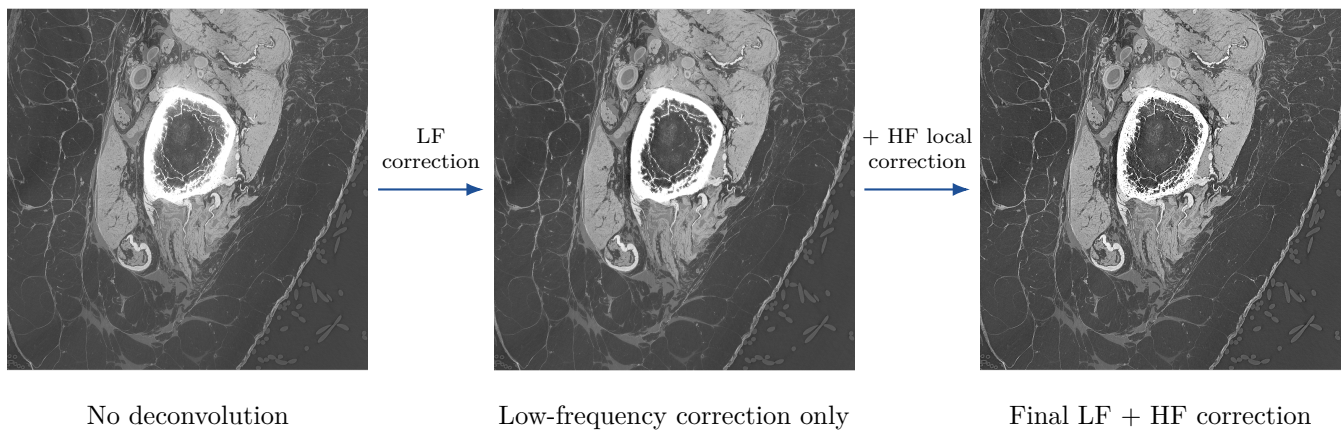


FIGURE 11.1 – Effet progressif de la correction sur une même coupe reconstruite. De gauche à droite : sans déconvolution, avec correction basse fréquence seule, puis avec la correction finale LF + HF. Les trois images utilisent le même recadrage et la même plage de niveaux de gris.

Dans la première image, aucune déconvolution n'est appliquée. Elle correspond donc à la reconstruction obtenue avant correction de la diffusion optique.

Dans la deuxième image, seule la composante basse fréquence est appliquée. Une partie de l'effet de la correction est déjà retrouvée, mais cette composante contient volontairement uniquement la contribution lisse et de longue portée.

La troisième image correspond à la méthode développée pendant le stage. La composante basse fréquence est calculée à résolution spatiale et angulaire réduite, puis la composante haute fréquence locale est ajoutée séparément. Cette recombinaison constitue la reconstruction finale obtenue avec la nouvelle stratégie.

11.2 Comparaison avec la correction complète de référence

La comparaison la plus importante est celle entre la nouvelle méthode multirésolution et la correction complète utilisée comme référence.

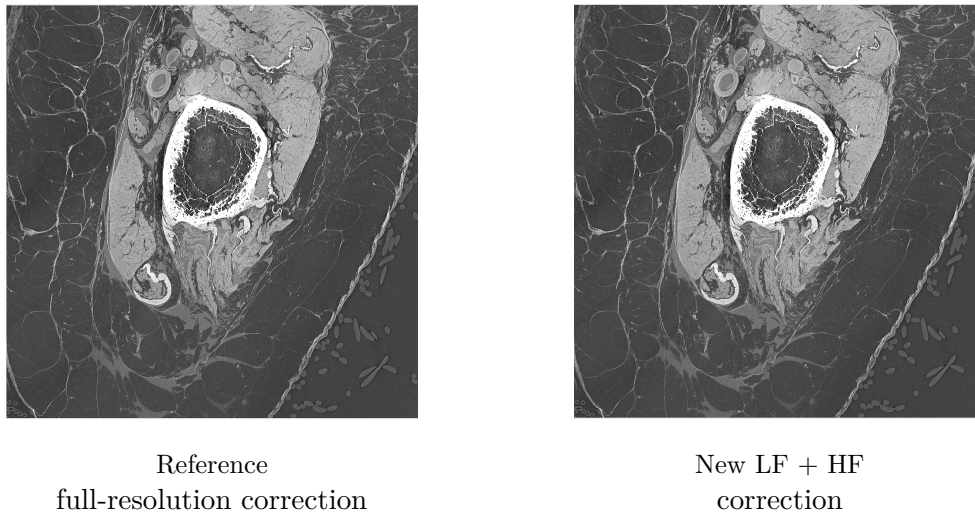


FIGURE 11.2 – Comparaison entre la reconstruction obtenue avec la correction complète de référence et celle obtenue avec la nouvelle stratégie LF + HF. Les deux images correspondent à la même coupe reconstruite, avec le même recadrage et la même plage de niveaux de gris, ce qui permet une comparaison directe.

L'objectif de la nouvelle méthode n'est pas de produire une correction différente, mais de retrouver aussi fidèlement que possible l'effet de la correction complète tout en évitant son coût de calcul et de stockage à pleine résolution.

Visuellement, la reconstruction obtenue avec la stratégie LF + HF reste très proche de celle obtenue avec la correction complète de référence. Les principales structures de la coupe sont conservées et aucune dégradation évidente n'apparaît dans cette comparaison.

Cette comparaison doit cependant être interprétée comme une validation visuelle. Les métriques quantitatives présentées dans les sections précédentes ont été calculées sur les composantes de correction et non directement entre ces deux images reconstruites.

Une fois vérifié que la nouvelle stratégie reste visuellement cohérente avec la correction de référence, la dernière question concerne donc le gain obtenu : quelle réduction apporte-t-elle réellement en termes d'espace disque et de temps de traitement ?

12 Performances mesurées

Après avoir vérifié que la nouvelle correction reste visuellement cohérente avec la correction complète de référence, il reste à évaluer le gain pratique apporté par cette nouvelle organisation du pipeline.

Les mesures finales ont été réalisées pour la configuration étudiée sur un nœud de calcul de 96 cœurs. Elles concernent trois aspects importants : l'espace disque nécessaire, le temps de création du digest et le temps de reconstruction.

TABLE 12.1 – Performances mesurées pour la méthode de référence et la nouvelle stratégie multirésolution.

Grandeur	Référence	Nouvelle méthode	Gain
Empreinte disque	4.7 To	80 Go	$\approx 59\times$ plus faible
Création du digest	32 min	4 min	$8\times$ plus rapide
Reconstruction	118 min	93 min	$\approx 21.2\%$ plus rapide

Le gain le plus important concerne l’empreinte disque. Le volume de données intermédiaires passe d’environ 4.7 To à 80 Go, soit une réduction d’un facteur proche de 59.

Le temps nécessaire à la création du digest passe quant à lui de 32 minutes à 4 minutes. Cette étape est donc environ huit fois plus rapide avec la nouvelle organisation.

Enfin, le temps de reconstruction complet diminue de 118 minutes à 93 minutes, soit un gain de 25 minutes et une réduction d’environ 21.2%.

Ces résultats montrent que la stratégie multirésolution ne constitue pas uniquement une réorganisation numérique de la correction. La séparation entre une composante basse fréquence réduite et une composante haute fréquence locale entraîne également une diminution concrète du coût du pipeline, aussi bien en termes de stockage que de temps de traitement.

Ces valeurs correspondent à la configuration, au jeu de données et au matériel utilisés pour ces mesures. Elles ne doivent donc pas être interprétées comme des performances universelles.

13 Outils utilisés et compétences développées

13.1 Environnement et outils utilisés

Le stage m’a amenée à utiliser un ensemble assez large d’outils, allant du calcul scientifique à l’édition de code, la visualisation d’images, la gestion de fichiers et l’intégration dans la chaîne de reconstruction de l’ESRF.

13.1.1 Programmation et calcul scientifique

- **Python** pour le prototypage, les tests et le traitement basse fréquence ;
- **NumPy** pour la manipulation des tableaux et des données ;
- **SciPy** pour le filtrage, l’interpolation et certaines opérations numériques ;
- **h5py** pour la lecture et l’écriture des fichiers HDF5 ;
- **pyFFTW** et **FFTW** pour les convolutions par transformée de Fourier ;
- **Matplotlib** pour les figures, profils et validations ;
- **multiprocessing** pour certains calculs parallélisés ;
- **C++** pour l’intégration de la composante haute fréquence dans **nabuxx** ;
- **xtensor** pour la manipulation des tableaux numériques côté C++ ;
- **HighFive** pour l’accès aux fichiers HDF5 côté C++.

13.1.2 Visualisation et traitement d’images

- **ImageJ** pour l’inspection et la comparaison des images et reconstructions ;
- **silx** pour la visualisation de données scientifiques et de fichiers HDF5 ;
- analyse de fichiers **.tif**, profils d’intensité, comparaison de régions d’intérêt et contrôle visuel des reconstructions.

13.1.3 Édition et comparaison de fichiers

- **Emacs** et **nano** pour l’édition de scripts et de fichiers directement sur les machines de calcul ;
- **Meld** pour comparer différentes versions de fichiers et identifier précisément les modifications introduites dans le code ;
- navigation et modification de fichiers en environnement Linux.

13.1.4 Environnements de calcul

- utilisation d’environnements virtuels Python afin d’isoler les dépendances nécessaires aux différents scripts ;
- installation et gestion de bibliothèques scientifiques ;
- travail sur des machines Linux distantes ;
- lancement et suivi de calculs depuis le terminal ;
- gestion de données expérimentales volumineuses et de fichiers intermédiaires.

13.1.5 Terminal Linux

Au cours du stage, j’ai progressivement utilisé le terminal comme un outil de travail quotidien. Parmi les commandes fréquemment utilisées :

```
pwd
ls
cd
cp
mv
rm
mkdir
grep
find
head
tail
less
ps
```

Ces commandes ont été utilisées pour naviguer dans les répertoires, rechercher des fichiers ou des portions de code, inspecter des résultats, organiser les données et suivre l’exécution des programmes.

13.1.6 Versionnement et gestion du code

- utilisation de **Git** pour suivre les modifications ;
- commandes telles que `git status`, `git diff`, `git add` et `git commit` ;
- utilisation de **GitHub** pour centraliser les scripts, figures et documents partageables liés au projet.

13.2 Compétences développées

13.2.1 Relier physique et calcul numérique

L’un des principaux acquis du stage a été d’apprendre à partir d’un phénomène physique réel pour construire progressivement une stratégie numérique.

Le travail a commencé par la diffusion optique dans le système de détection, puis par sa représentation sous forme de kernel et de convolution. L’étude des différentes échelles spatiales a ensuite conduit à la séparation basse fréquence / haute fréquence, puis à deux traitements numériques adaptés aux propriétés de chacune de ces composantes.

L’optimisation n’a donc pas été introduite comme une simple astuce de programmation : elle découle directement de la compréhension de la structure physique et spatiale de la correction.

13.2.2 Construire et valider une méthode progressivement

Le développement a également demandé une démarche de validation progressive. Avant de modifier la chaîne complète, chaque nouvelle idée a été testée sur des cas contrôlés : binning, interpolation entre projections, séparation fréquentielle, taille de la marge haute fréquence puis recombinaison des deux branches.

Cette méthode m’a appris à ne pas juger une optimisation uniquement sur son temps d’exécution, mais à vérifier d’abord l’erreur qu’elle introduit et à identifier précisément la composante qui devient limitante.

13.2.3 Manipuler des données expérimentales volumineuses

Le projet m’a confrontée à des volumes de données beaucoup plus importants que dans mes projets universitaires précédents. Cela m’a amenée à prendre en compte non seulement le temps de calcul, mais aussi l’espace disque, les accès aux fichiers, les allocations mémoire, la réutilisation des kernels et le coût des données intermédiaires.

Cette expérience a donné une dimension plus concrète à la notion de performance en calcul scientifique.

13.2.4 Intégrer une solution dans un environnement de production

Une autre compétence importante a été de passer du prototype à l’intégration.

Une solution qui fonctionne dans un script isolé ne suffit pas lorsqu’elle doit être insérée dans une chaîne utilisée pour traiter de véritables acquisitions. Il faut gérer les formats de fichiers, les paramètres, les dimensions, les blocs de projections, les interfaces Python/C++ et la transmissibilité du code.

Cette étape a constitué ma première expérience significative d’intégration dans une base C++ scientifique existante.

13.2.5 Communication scientifique

Enfin, ce stage m’a également demandé de synthétiser un travail mêlant physique, traitement d’image et informatique scientifique afin de le rendre compréhensible à différents interlocuteurs.

La préparation du rapport et de la présentation finale m’a notamment obligée à distinguer les éléments essentiels du projet des détails d’implémentation, et à construire une explication allant du phénomène physique jusqu’aux gains mesurés.

Conclusion

Ce stage à l'ESRF m'a permis de découvrir concrètement la manière dont un problème physique peut être transformé en problème numérique, puis intégré dans une chaîne de traitement utilisée sur des données expérimentales réelles.

Le travail réalisé m'a amenée à combiner plusieurs aspects que j'avais jusqu'alors abordés séparément : compréhension physique du système de détection, traitement d'image, méthodes numériques, calcul scientifique, gestion de grands volumes de données et développement dans une base de code existante.

J'ai également acquis une expérience beaucoup plus pratique des environnements de recherche informatique : Linux, travail sur machines distantes, Python, C++, FFTW, HDF5, Git, outils de visualisation et débogage. Une partie importante du stage a consisté à apprendre à tester une idée progressivement, à identifier précisément l'origine d'une erreur et à vérifier qu'une optimisation restait cohérente avec le résultat physique attendu.

Ce stage s'inscrit dans la continuité de mon parcours scientifique. Après une première expérience à l'IRAM consacrée à l'analyse de données interférométriques ALMA, l'ESRF m'a permis de découvrir un autre grand instrument scientifique et de travailler davantage à l'interface entre physique, instrumentation et calcul.

À la suite de cette expérience, j'intègre le Master Nanophysics and Quantum Physics de l'Université Grenoble Alpes ainsi que le Quantum Program de la Graduate School. Je souhaite poursuivre dans des projets combinant physique, instrumentation, traitement numérique et technologies quantiques, en m'appuyant sur les compétences développées progressivement au cours de mes expériences à l'IRAM puis à l'ESRF.

A Fichiers, scripts et dépôt du projet

Les principaux fichiers de production utilisés au cours du stage sont :

- `diffusion_correction.py`;
- `diffusion_parameters.json`;
- `diffusion_correction.h5`;
- `kernel.h5`;
- `etfscan.cpp`;
- `etfscan.hpp`.

Plusieurs scripts indépendants ont également été développés pendant la phase de validation afin d'étudier successivement le binning, les interpolations, la séparation basse fréquence / haute fréquence, la correction locale et la recombinaison multirésolution.

Le script :

`23_full_correction_spatial4_angular4_local_high.py`

constitue notamment une synthèse de la phase de validation du pipeline.

L'ensemble des éléments pouvant être partagés, notamment les scripts de validation, les figures et les documents de présentation du projet, est regroupé dans le dépôt GitHub associé au stage.

Dépôt GitHub :

<https://github.com/alouaniranou2004-droid>